Q1 (a) Define Software Quality. List & Explain Core Components of Quality. [8 Marks]

Definition of Software Quality

Software Quality is the degree to which a software product conforms to specified requirements, satisfies customer needs and expectations, and is free from defects while being reliable, efficient, secure, maintainable, and usable.

IEEE Definition:

Software quality is the degree to which a software system, component, or process meets specified requirements and customer expectations.

Core Components of Quality

The four core components of quality are **Customer, Supplier, Process, and Product**.

1. Customer

- The customer is the end user or organization that uses the software.
- Customer requirements define the expected quality of the software.
- Customer satisfaction is the primary objective of quality management.

Example: A banking customer expects secure and error-free online transactions.

---

2. Supplier

- The supplier is the software development organization or team that develops and delivers the software.
- The supplier is responsible for following quality standards, coding practices, and testing procedures.
- A skilled supplier ensures a high-quality product.

Example: A software company developing an e-commerce website.

---

3. Process

- A process is a structured set of activities followed during software development.
- It includes requirement analysis, design, coding, testing, deployment, and maintenance.
- A well-defined process reduces defects and improves consistency.

Example: Following the Software Development Life Cycle (SDLC) ensures systematic development.

---

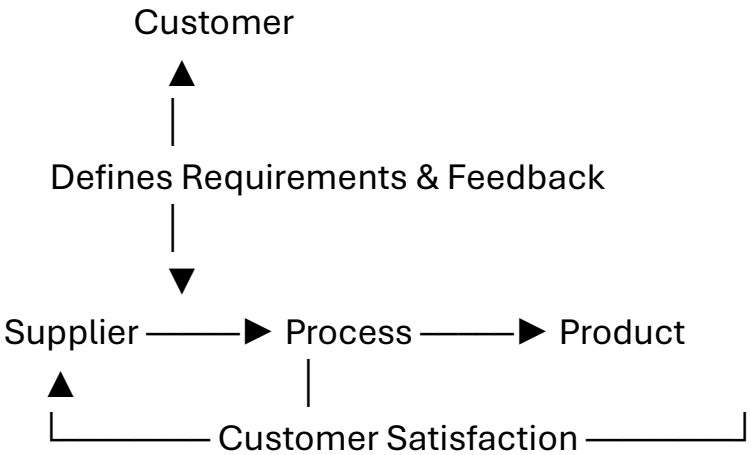
4. Product

- The product is the final software delivered to the customer.
- It should meet both functional and non-functional requirements.

- A quality product is:
    - Correct
    - Reliable
    - Secure
    - Efficient
    - Easy to maintain
    - User-friendly

Example: A mobile banking application that performs transactions accurately and securely.

Relationship Between Core Components



Explanation:

- The **Customer** specifies requirements.
- The **Supplier** follows a proper **Process** to develop the software.
- The **Process** produces the **Product**.
- The **Product** is delivered to the **Customer**, whose feedback helps improve future quality.

Importance of Core Components

- Ensures customer satisfaction.
- Reduces software defects.
- Improves software reliability.
- Enhances maintainability.
- Delivers software on time and within budget.
- Supports continuous quality improvement.

Conclusion

Software quality is achieved when a **supplier** follows a well-defined **process** to develop a **product** that fulfills **customer** requirements. These four core components work together to produce reliable, efficient, and high-quality software.

Q1 (b) What are the Constraints of Software Product Quality Assessment? [8 Marks]

Definition

Software Product Quality Assessment is the process of evaluating whether a software product meets specified requirements, quality standards, and customer expectations in terms of functionality, reliability, performance, usability, security, and maintainability.

However, assessing software quality is difficult due to several constraints.

Constraints of Software Product Quality Assessment

1. Intangible Nature of Software

- Software is an intangible product and cannot be physically inspected like hardware.
- Quality is judged through testing, reviews, and execution.

Example: A bug in the source code cannot be detected by simply looking at the software.

---

2. Changing Requirements

- Customer requirements may change during software development.
- Frequent changes make it difficult to maintain and assess quality.

Example: A customer requests new features after development has started.

---

3. Time Constraints

- Projects often have strict deadlines.
- Limited testing time may result in some defects remaining undetected.

---

4. Budget Constraints

- Limited financial resources restrict testing activities, tools, and manpower.
- This can reduce the overall quality assessment.

---

5. Complexity of Software

- Modern software systems contain millions of lines of code and multiple interacting modules.
- Higher complexity makes quality assessment more difficult.

---

6. Exhaustive Testing is Impossible

- Software can have an enormous number of input combinations and execution paths.
- Testing every possible case is practically impossible.

---

7. Human Errors

- Developers, testers, and analysts can make mistakes during development and testing.
- Human errors may introduce defects or overlook existing ones.

---

8. Different Operating Environments

- Software may behave differently on different operating systems, browsers, devices, or hardware configurations.
- Ensuring consistent quality across all environments is challenging.

---

9. Lack of Complete User Feedback

- Quality assessment before release may not reflect real-world usage.
- Actual users may discover defects that were not found during testing.

---

10. Measurement Difficulty

- Some quality attributes, such as usability, maintainability, and customer satisfaction, are difficult to measure objectively.

Q1 (c) Explain Defect Life Cycle with the Help of Diagram. [7 Marks]

Definition

A **Defect Life Cycle (Bug Life Cycle)** is the sequence of stages through which a defect passes from the time it is identified until it is fixed, verified, and finally closed. It helps track defects systematically and ensures that no reported defect is left unresolved.

Stages of Defect Life Cycle

1. New

- A tester identifies a defect and reports it in the defect tracking tool.

- The defect status is marked as **New**.

---

2. Assigned

- The project manager or test lead assigns the defect to the appropriate developer for fixing.

---

3. Open

- The developer analyzes the defect and starts working on it.
- The defect status changes to **Open**.

---

4. Fixed

- The developer corrects the defect and updates the status to **Fixed**.
- The software is sent back to the testing team.

---

5. Retest

- The tester retests the application to verify whether the defect has been fixed correctly.

---

6. Closed

- If the defect is successfully fixed and no issue is found during retesting, the tester changes the status to **Closed**.

Other Possible States

Reopened

- If the defect still exists after retesting, it is marked **Reopened** and sent back to the developer.

Rejected

- The developer determines that the reported issue is not a valid defect.

Deferred

- The defect is valid but its fix is postponed to a future release due to low priority or time constraints.

Duplicate

- The same defect has already been reported by another tester.

Cannot Reproduce

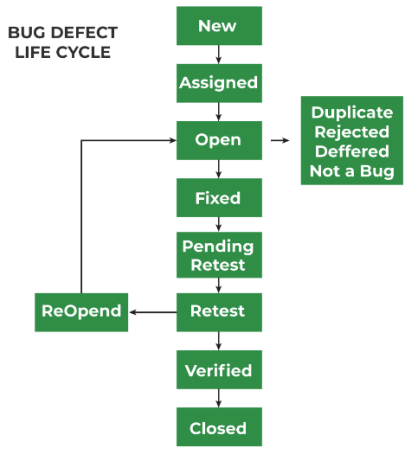
- The developer or tester is unable to reproduce the defect in the given environment.

Other States:

- Rejected
- Deferred
- Duplicate
- Cannot Reproduce

Advantages of Defect Life Cycle

- Tracks defects from detection to closure.
- Improves communication between testers and developers.
- Ensures every defect is properly resolved.
- Helps prioritize and manage defects efficiently.
- Improves the overall quality and reliability of the software.



Q2 (a) Examine the Relationship between Quality and Productivity. [7 Marks]

Definition

- **Quality** is the degree to which a software product meets customer requirements and is free from defects.
- **Productivity** is the measure of the amount of software developed using available resources (time, cost, and manpower).

Productivity Formula:

\textbf{Productivity = \frac{\text{Output}}{\text{Input}}}

Example:

- Output = Number of software modules developed
- Input = Time, cost, and effort

Relationship Between Quality and Productivity

Quality and productivity are **closely related**. High-quality software improves productivity because fewer defects require less rework, while poor quality decreases productivity due to repeated corrections.

1. High Quality Reduces Rework

- Defect-free software requires fewer corrections.
- Less time is spent fixing bugs.
- Developers can focus on new features.

---

2. Improved Productivity

- Fewer defects reduce testing and maintenance effort.
- Projects are completed faster and at lower cost.

---

3. Customer Satisfaction

- High-quality software satisfies customer requirements.
- Fewer complaints and change requests improve team productivity.

---

4. Lower Development Cost

- Early defect detection reduces the cost of fixing errors.
- Resources are used more efficiently.

---

5. Better Team Efficiency

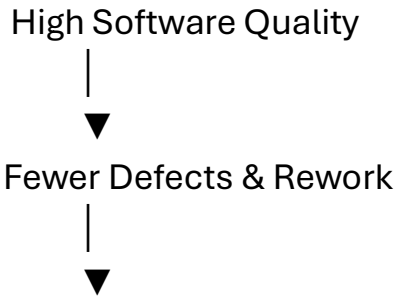
- Standard coding practices and quality processes reduce confusion.
- Teams work more effectively and deliver software on time.

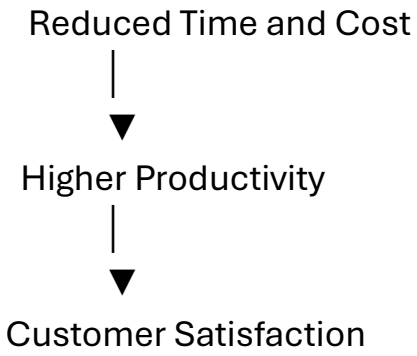
---

6. Continuous Improvement

- Quality management techniques such as **TQM (Total Quality Management)** and **Continuous Process Improvement** improve both quality and productivity over time.

Diagram





Advantages of High Quality on Productivity

- Produces reliable software.
- Reduces maintenance effort.
- Improves customer satisfaction.
- Saves development time and cost.
- Increases team efficiency.
- Delivers software faster.

Q2 (b) Explain PDCA Cycle. [7 Marks]

Definition

The **PDCA Cycle** (Plan–Do–Check–Act Cycle), also known as the **Deming Cycle** or **Shewhart Cycle**, is a continuous improvement model used in **Total Quality Management (TQM)**. It helps organizations improve the quality of products, processes, and services through an iterative four-step approach.

Phases of the PDCA Cycle

1. Plan (P)

- Identify the problem or opportunity for improvement.
- Analyze the current process.
- Set quality objectives and prepare an action plan.

Example: Identify frequent software defects and plan strategies to reduce them.

---

2. Do (D)

- Implement the planned solution on a small scale.
- Develop or modify the process.
- Collect data during implementation.

Example: Introduce code reviews and automated testing in one project.

---

3. Check (C)

- Compare actual results with expected results.
- Analyze whether the implemented solution achieved the desired improvement.
- Identify any deviations or remaining problems.

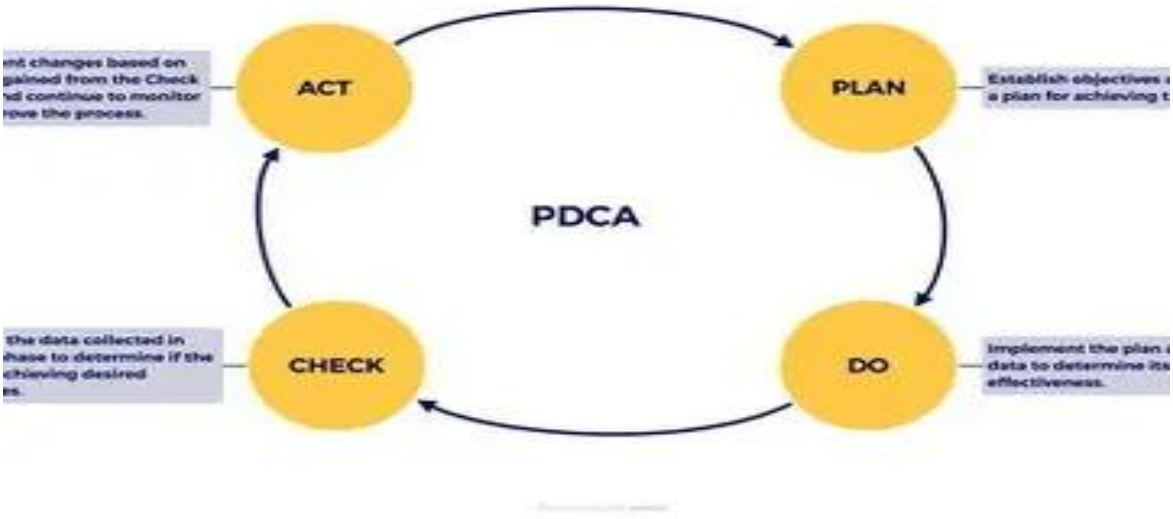
Example: Measure the reduction in software defects after implementing code reviews.

---

4. Act (A)

- If the solution is successful, standardize and implement it across the organization.
- If not successful, identify the causes and repeat the PDCA cycle with improvements.

Example: Make code reviews a mandatory practice for all software development projects.



Advantages of PDCA Cycle

- Ensures continuous quality improvement.
- Reduces defects and development costs.
- Improves productivity and efficiency.
- Helps in better decision-making.
- Increases customer satisfaction.
- Supports effective quality management.

Applications of PDCA in Software Testing

- Improving software development processes.
- Reducing software defects.

- Enhancing testing effectiveness.
- Monitoring project quality.
- Implementing process improvements.

Q2 (c) Plan Software Quality Control with Respect to College Attendance Software. [7 Marks]

Definition

Software Quality Control (SQC) is the process of identifying and removing defects from software through activities such as reviews, inspections, and testing to ensure that the software meets specified quality standards.

Software Quality Control Plan for College Attendance Software

1. Requirement Verification

- Verify that all functional and non-functional requirements are clearly defined.
- Ensure features like student login, faculty login, attendance marking, report generation, and notifications are included.

---

2. Design Review

- Review the database design, user interface, and system architecture.
- Ensure the attendance records are stored correctly and securely.

---

3. Code Review

- Conduct peer reviews of the source code.
- Follow coding standards to reduce defects and improve maintainability.

---

4. Testing Activities

- **Unit Testing:** Test individual modules such as Login, Attendance Entry, and Report Generation.
- **Integration Testing:** Verify interaction between modules (e.g., Login → Attendance → Database).
- **System Testing:** Test the complete attendance system.
- **Acceptance Testing:** Ensure the software satisfies college requirements.

---

5. Defect Management

- Record all defects in a defect tracking system.
- Assign defects to developers, fix them, retest, and close them.

---

6. Performance and Security Testing

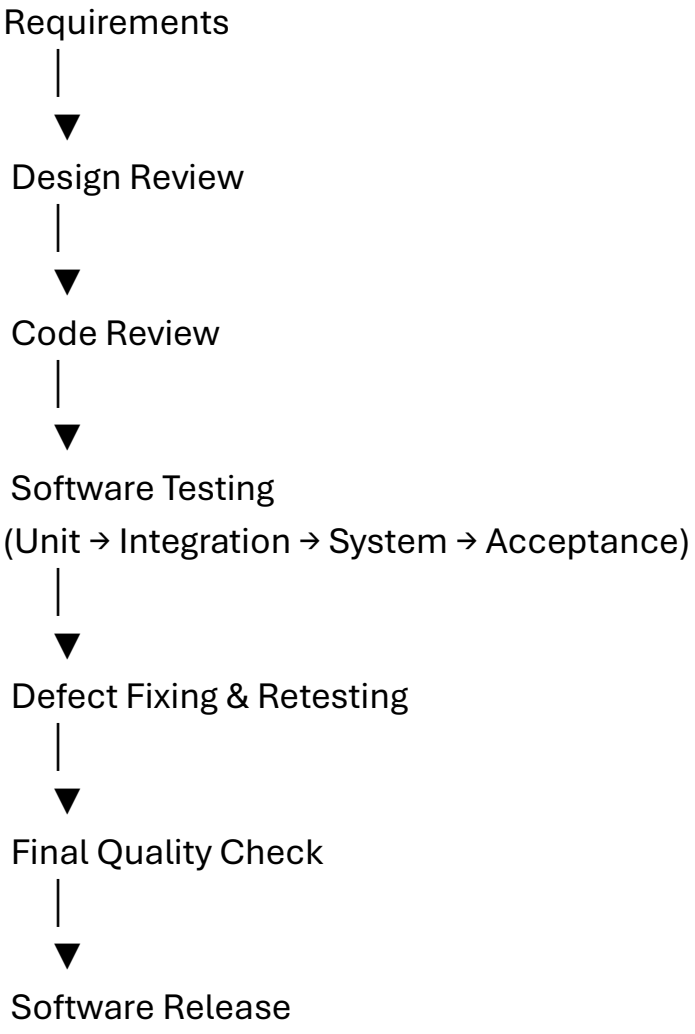
- Check system performance with multiple users.
- Ensure only authorized users (admin, faculty, students) can access the system.
- Protect attendance records from unauthorized modifications.

---

7. Documentation and Maintenance

- Prepare user manuals and testing reports.
- Maintain software by fixing bugs and updating features when required.

Software Quality Control Flow Diagram



Expected Quality Attributes

- Accurate attendance calculation.
- Secure login and data protection.
- Fast response time.
- User-friendly interface.
- Reliable report generation.
- Easy maintenance and updates.

Q3 (a) Define "Quality" as Viewed by Different Stakeholders of Software Development and Usage. [7 Marks]

Definition of Quality

Quality is the degree to which a software product satisfies specified requirements, fulfills customer expectations, and is free from defects. Different stakeholders view quality from different perspectives based on their roles and responsibilities.

Quality as Viewed by Different Stakeholders

1. Customer (End User)

- Customers consider quality as software that meets their requirements and is easy to use.
- They expect reliability, security, correctness, and good performance.

Example: An online banking application should perform transactions accurately and securely.

---

2. Software Developer

- Developers view quality as software that is well-designed, error-free, maintainable, and follows coding standards.
- They focus on writing efficient and reusable code.

---

3. Tester (Quality Assurance Team)

- Testers define quality as software with minimum defects that satisfies all functional and non-functional requirements.
- They verify quality through testing and defect detection.

---

4. Project Manager

- Project managers consider quality as delivering software **on time, within budget, and according to specifications**.
- They focus on resource management and customer satisfaction.

---

5. Organization (Management)

- Management views quality as delivering reliable software while reducing development and maintenance costs.
- High-quality software improves the company's reputation and customer trust.

---

6. Maintenance Team

- The maintenance team considers quality as software that is easy to modify, update, and debug.
- Good documentation and modular design improve maintainability.

Q3 (b) Describe "Total Quality Management (TQM)" Principles of Continual Improvement. [7 Marks]

Definition of Total Quality Management (TQM)

Total Quality Management (TQM) is a management approach that focuses on **continuous improvement of products, services, and processes** by involving all employees to achieve **customer satisfaction**.

Principle of Continual Improvement

Continual Improvement is one of the key principles of TQM. It means that an organization should continuously improve its processes, products, and services instead of being satisfied with current performance.

The goal is to reduce defects, improve efficiency, lower costs, and increase customer satisfaction.

Principles of Continual Improvement

1. Customer Focus

- Understand customer needs and expectations.
- Improve software based on customer feedback.
- Aim for maximum customer satisfaction.

---

2. Employee Involvement

- Every employee should participate in quality improvement.
- Teamwork and training improve overall performance.

---

3. Process Improvement

- Analyze existing processes regularly.
- Remove unnecessary activities and improve efficiency.

---

4. Defect Prevention

- Focus on preventing defects rather than correcting them later.
- Early reviews and testing reduce development costs.

---

5. Measurement and Analysis

- Measure quality using metrics such as:
  - Defect density
  - Customer satisfaction
  - Productivity
  - Testing efficiency
- Use collected data for further improvements.

---

6. PDCA (Plan–Do–Check–Act) Cycle

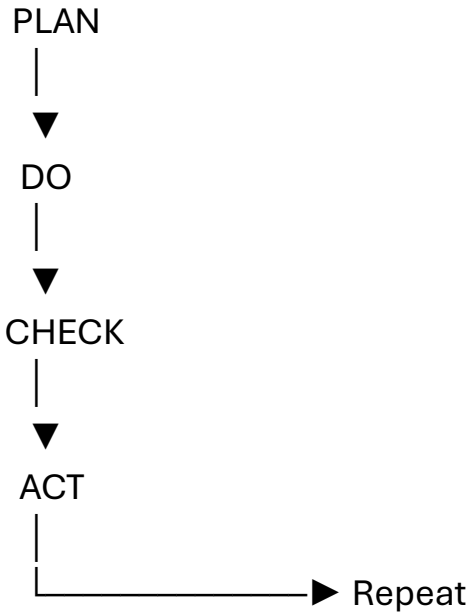
- Follow the PDCA cycle continuously:
  - **Plan:** Identify improvement opportunities.
  - **Do:** Implement changes.
  - **Check:** Evaluate the results.
  - **Act:** Standardize successful improvements.

---

7. Continuous Learning

- Learn from previous projects and defects.
- Apply lessons learned to future software development.

Continual Improvement Cycle



Benefits of Continual Improvement

- Improves software quality.
- Reduces defects and rework.
- Increases productivity.

- Lowers development and maintenance costs.
- Enhances customer satisfaction.
- Improves team performance and efficiency.

Q2 (a) Give Classification for Different Types of Products. [7 Marks]

Definition

A **software product** is a software application developed to satisfy the requirements of users or organizations. Software products can be classified based on their purpose, users, and method of development.

Classification of Different Types of Products

1. Generic Products (Packaged Products)

- Developed for the general market.
- Sold to many customers.
- Features are decided by the software company.

Examples:

- Microsoft Office
- Windows
- Adobe Photoshop
- VLC Media Player

---

2. Customized (Bespoke) Products

- Developed for a specific customer or organization.
- Features are based on customer requirements.
- Cannot be sold to the general market.

Examples:

- College Attendance Management System
- Hospital Management System
- Banking Software
- Railway Reservation System

---

3. System Software

- Controls and manages computer hardware.
- Provides a platform for application software.

Examples:

- Operating Systems (Windows, Linux)

- Device Drivers
- Compilers
- Assemblers

---

4. Application Software

- Designed to perform specific user tasks.
- Runs on top of system software.

Examples:

- MS Word
- Excel
- Web Browsers
- WhatsApp

---

5. Embedded Software

- Installed inside hardware devices.
- Performs dedicated functions.

Examples:

- ATM Machine Software
- Smart TV Software
- Washing Machine Controller
- Car Engine Control System

---

6. Web-Based Software

- Runs on web browsers.
- Accessible through the Internet.

Examples:

- Gmail
- Amazon
- Online Banking
- College ERP Portal

---

7. Mobile Applications

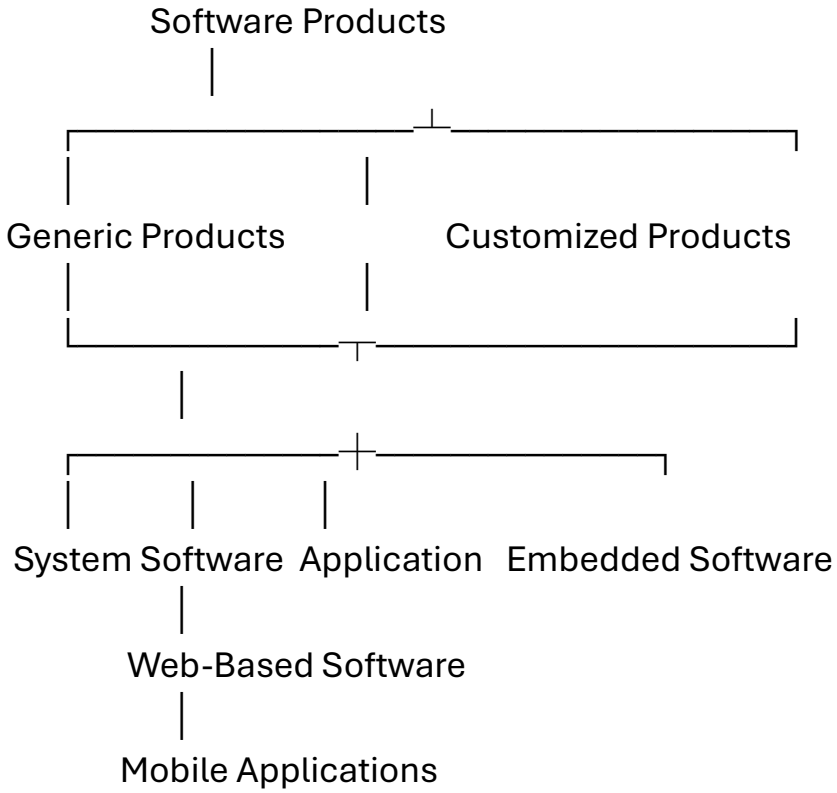
- Developed for smartphones and tablets.

Examples:

- Instagram
- Google Maps
- PhonePe

- Paytm

Classification Diagram



Advantages of Product Classification

- Helps choose the appropriate development process.
- Identifies customer requirements.
- Improves quality management.
- Simplifies software maintenance.
- Assists in project planning and testing.

Q2 (b) Plan Software Quality Control with Respect to Space Research. [7 Marks]
Definition

Software Quality Control (SQC) is the process of ensuring that software is free from defects and meets the required quality standards through reviews, inspections, testing, and verification.

In **space research**, software quality is extremely critical because even a small software error can lead to mission failure.

Software Quality Control Plan for Space Research

1. Requirement Analysis

- Clearly define mission requirements.

- Verify functional, performance, safety, and reliability requirements.

---

2. Design Review

- Review software architecture, algorithms, and database design.
- Ensure fault tolerance and high reliability.

---

3. Code Review

- Conduct peer reviews and inspections.
- Follow coding standards to minimize defects.

---

4. Testing

Perform different levels of testing:

- **Unit Testing** – Test each software module.
- **Integration Testing** – Verify interaction between modules.
- **System Testing** – Test the complete system.
- **Acceptance Testing** – Ensure software meets mission requirements.

---

5. Performance & Reliability Testing

- Verify software under extreme conditions.
- Test for accuracy, stability, and continuous operation.

---

6. Defect Management

- Record, analyze, fix, and retest defects.
- Use defect tracking tools to monitor defect status.

---

7. Documentation & Maintenance

- Maintain detailed documentation of requirements, test cases, and defect reports.
- Update software whenever required after deployment.

Quality Control Flow

Requirements

Design Review

Code Review

Testing
(Unit → Integration → System → Acceptance)

Defect Fixing & Retesting

Final Quality Check

Software Deployment

Conclusion
A well-planned quality control process ensures that space research software is **accurate, reliable, safe, and defect-free**, reducing the risk of mission failure.

Q2 (c) Compare Between Tools and Techniques. [7 Marks]
Definition

- **Tools** are software applications that automate testing and quality assurance activities.
- **Techniques** are systematic methods or approaches used to identify defects and improve software quality.

Difference Between Tools and Techniques

Basis	Tools	Techniques
Definition	Software used to automate testing activities	Methods used to perform testing and quality assurance
Purpose	Automate testing and improve efficiency	Detect defects and improve software quality
Nature	Software or application	Procedure or methodology
Execution	Performed using software	Performed manually or systematically
Cost	Usually requires purchase or installation	Mostly knowledge-based and low cost

Basis	Tools	Techniques
Examples	Selenium, JUnit, LoadRunner, JIRA	Black Box Testing, White Box Testing, Boundary Value Analysis, Equivalence Partitioning
Outcome	Faster execution and reporting	Better test case design and defect detection

---

Examples

Software Testing Tools

- Selenium
- JUnit
- TestNG
- LoadRunner
- JIRA

Software Testing Techniques

- Black Box Testing
- White Box Testing
- Grey Box Testing
- Boundary Value Analysis (BVA)
- Equivalence Partitioning (EP)
- Decision Table Testing

Conclusion

Tools help automate and speed up software testing, while **techniques** provide the methods for designing effective test cases and detecting defects. Both are essential for delivering high-quality software.

Q1 (a) Construct SDLC for Any Real-Life Application. [7 Marks]

Definition

Software Development Life Cycle (SDLC) is a systematic process used to develop high-quality software by following a sequence of well-defined phases from planning to maintenance.

Real-Life Application: Online Shopping System (Amazon/Flipkart)

Phases of SDLC

1. Requirement Analysis

- Gather requirements from customers and stakeholders.

- Identify functional and non-functional requirements.

Requirements:

- User Registration/Login
- Product Search
- Add to Cart
- Online Payment
- Order Tracking
- Admin Panel

---

2. Planning

- Estimate project cost, time, and resources.
- Prepare project schedule and assign team members.
- Identify project risks.

---

3. System Design

- Design the system architecture.
- Design database, user interface, and modules.
- Prepare ER diagrams, UML diagrams, and flowcharts.

---

4. Implementation (Coding)

- Develop software modules using programming languages.
- Integrate frontend, backend, and database.

Example Technologies:

- Frontend: HTML, CSS, JavaScript, React
- Backend: Java, Python, Node.js
- Database: MySQL, MongoDB

---

5. Testing

- Verify that all modules work correctly.
- Detect and fix defects before deployment.

Types of Testing:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

---

6. Deployment

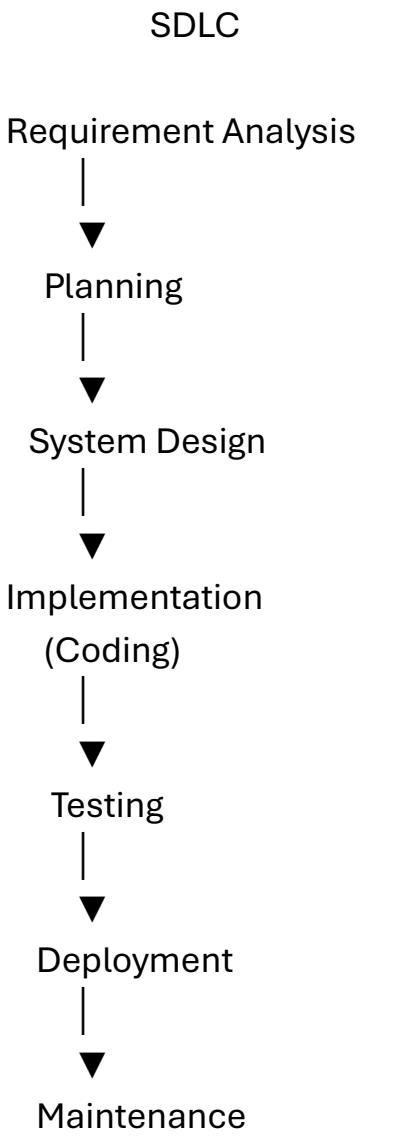
- Install the application on the production server.
- Make it available to customers.

---

7. Maintenance

- Fix bugs.
- Improve performance.
- Add new features.
- Update security patches.

SDLC Diagram



Advantages of SDLC

- Produces high-quality software.
- Reduces development cost and time.
- Improves project management.

- Detects defects early.
- Enhances customer satisfaction.
- Simplifies software maintenance.

Q1 (b) Write a Note on Pillars of Quality Management System (QMS). [7 Marks]
Definition
A **Quality Management System (QMS)** is a set of **policies, processes, procedures, and resources** used by an organization to ensure that products and services consistently meet customer requirements and quality standards while achieving continuous improvement.

Pillars of Quality Management System (QMS)
1. Customer Focus

- Customer satisfaction is the primary objective of QMS.
- Understand customer needs and deliver products that meet or exceed their expectations.
- Collect customer feedback for continuous improvement.

---

2. Leadership

- Top management establishes the organization's vision, quality policy, and objectives.
- Leaders motivate employees and create a culture focused on quality.

---

3. Employee Involvement

- Every employee contributes to quality improvement.
- Proper training, teamwork, and participation help reduce defects and improve productivity.

---

4. Process Approach

- Manage activities as interconnected processes rather than separate tasks.
- A systematic process improves efficiency, consistency, and product quality.

---

5. Continual Improvement

- Continuously improve products, services, and processes.
- Use methods such as the **PDCA (Plan–Do–Check–Act) Cycle** to enhance quality.

6. Evidence-Based Decision Making

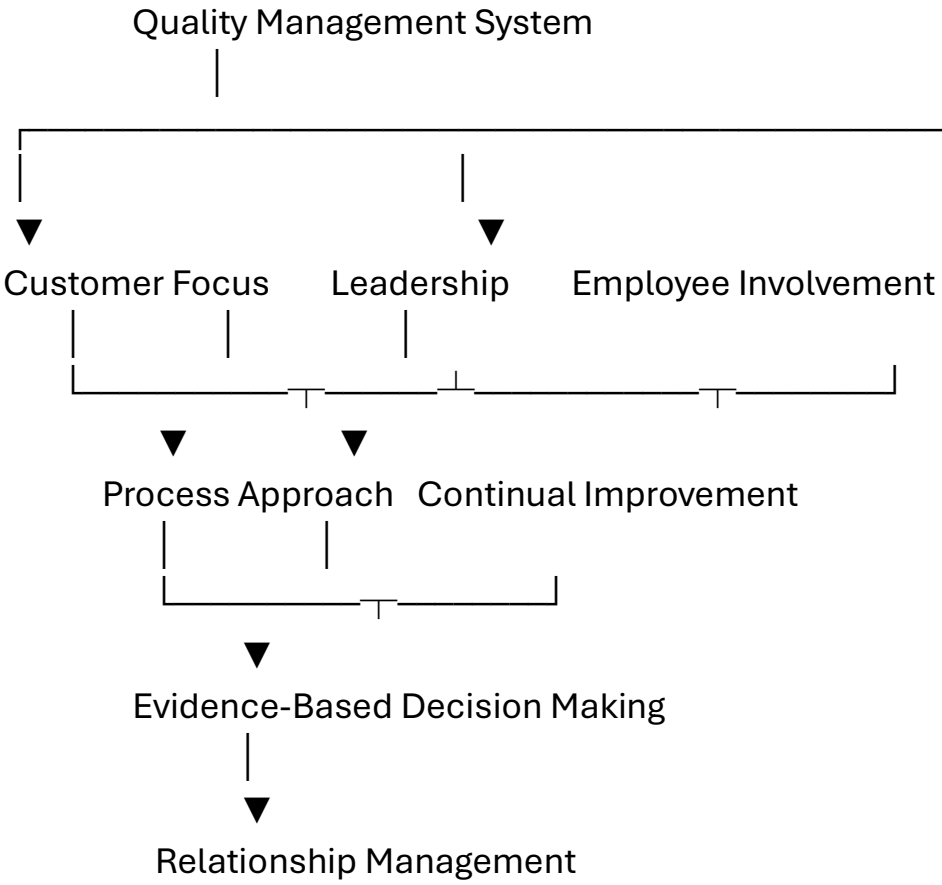
- Decisions should be based on data, testing results, measurements, and analysis.
- Accurate information helps improve software quality and reduce risks.

---

7. Relationship Management

- Maintain good relationships with customers, suppliers, employees, and other stakeholders.
- Strong relationships improve communication, quality, and long-term success.

Pillars of QMS Diagram



Benefits of QMS

- Improves software quality.
- Increases customer satisfaction.
- Reduces defects and rework.
- Improves productivity and efficiency.
- Ensures continuous improvement.
- Helps deliver software on time and within budget.

Conclusion

The **seven pillars of Quality Management System—Customer Focus, Leadership, Employee Involvement, Process Approach, Continual Improvement, Evidence-Based Decision Making, and Relationship Management**—help organizations develop high-quality software, improve customer satisfaction, and achieve long-term business success.

Q1 (c) What is the Difference Between Testing and Debugging? [7 Marks]

Definition

Testing

Testing is the process of executing a software application to identify defects, errors, or missing requirements and to verify that the software works as expected.

Debugging

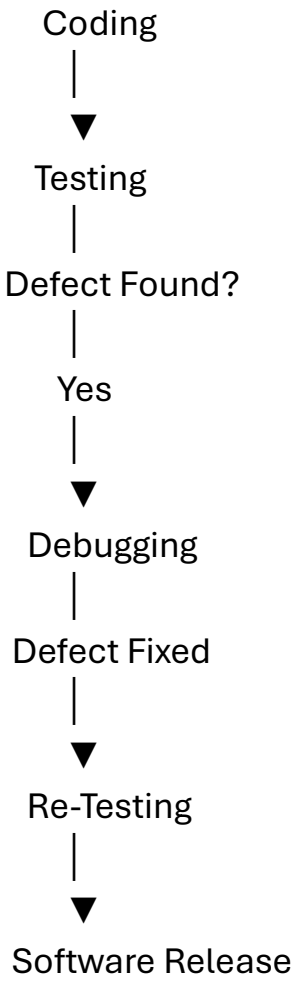
Debugging is the process of identifying the root cause of defects found during testing and correcting them in the source code.

Difference Between Testing and Debugging

Basis	Testing	Debugging
Definition	Process of finding defects in software	Process of locating and fixing defects
Objective	Detect errors and verify software quality	Remove errors and correct the software
Performed By	Testers / QA Engineers	Developers / Programmers
When Performed	After coding or during development	After defects are reported during testing
Knowledge Required	Functional and requirement knowledge	Programming and code knowledge
Output	Defect/Bug Report	Corrected and updated software
Tools Used	Selenium, JUnit, TestNG, LoadRunner	Debugger, GDB, Visual Studio Debugger, Eclipse Debugger

---

Testing and Debugging Process



Advantages of Testing

- Detects defects before software release.
- Improves software quality.
- Ensures software meets customer requirements.
- Increases reliability and security.

Advantages of Debugging

- Removes software defects.
- Improves software performance.
- Makes software stable and reliable.
- Prevents the recurrence of similar defects.

Q2 (a) Explain the Problematic Areas of Software Development Life Cycle (SDLC). [7 Marks]

Definition

The **Software Development Life Cycle (SDLC)** is a structured process used to develop software systematically. However, various challenges or problematic areas can arise during different phases of the SDLC, affecting software quality, cost, and project completion.

Problematic Areas of SDLC

1. Changing Requirements

- Customer requirements may change during development.
- Frequent changes increase development time and cost.

Example: Adding new features after coding has started.

---

2. Poor Requirement Analysis

- Incomplete or unclear requirements lead to misunderstandings.
- Results in incorrect software development.

---

3. Poor Project Planning

- Incorrect estimation of time, cost, and resources.
- May cause project delays and budget overruns.

---

4. Design Complexity

- Poor software architecture or design makes the system difficult to understand and maintain.
- Increases the possibility of defects.

---

5. Coding Errors

- Developers may introduce logical, syntax, or runtime errors.
- Poor coding standards reduce software quality.

---

6. Inadequate Testing

- Insufficient testing leaves defects undetected.
- May result in software failures after deployment.

---

7. Communication Gap

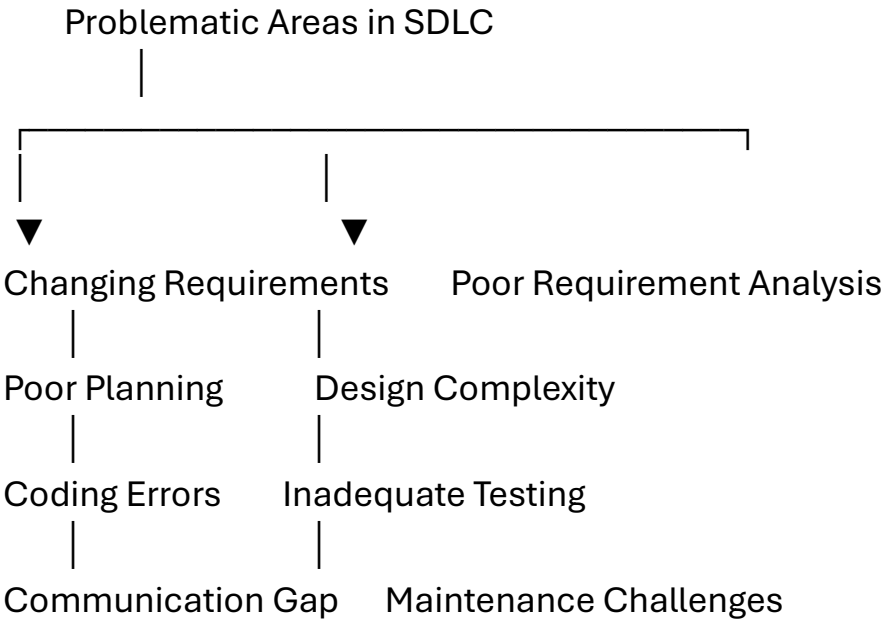
- Poor communication between customers, developers, testers, and managers causes misunderstandings and delays.

---

8. Maintenance Challenges

- Fixing bugs and adding new features after deployment can be difficult if documentation is poor.
- Maintenance increases overall software cost.

Diagram



Effects of These Problems

- Increase in project cost.
- Delay in project completion.
- Low software quality.
- Customer dissatisfaction.
- Increased maintenance effort.
- Higher risk of software failure.

Q2 (b) List and Explain Quality Objectives Applicable to Software Development and Usage. [7 Marks]

Definition

Quality Objectives are measurable goals established by an organization to ensure that software products meet customer requirements, improve performance, and achieve continuous quality improvement.

Quality Objectives

1. Customer Satisfaction

- Develop software that fulfills customer requirements and expectations.
- Collect customer feedback for continuous improvement.

---

2. Defect Prevention

- Prevent defects instead of correcting them after development.
- Use reviews, inspections, and testing to minimize errors.

---

3. Reliability

- Ensure the software performs consistently without failures.
- Increase system availability and dependability.

---

4. Performance and Efficiency

- Software should respond quickly and use system resources efficiently.
- Reduce execution time and improve speed.

---

5. Maintainability

- Software should be easy to modify, update, and debug.
- Good documentation and modular design improve maintainability.

---

6. Security

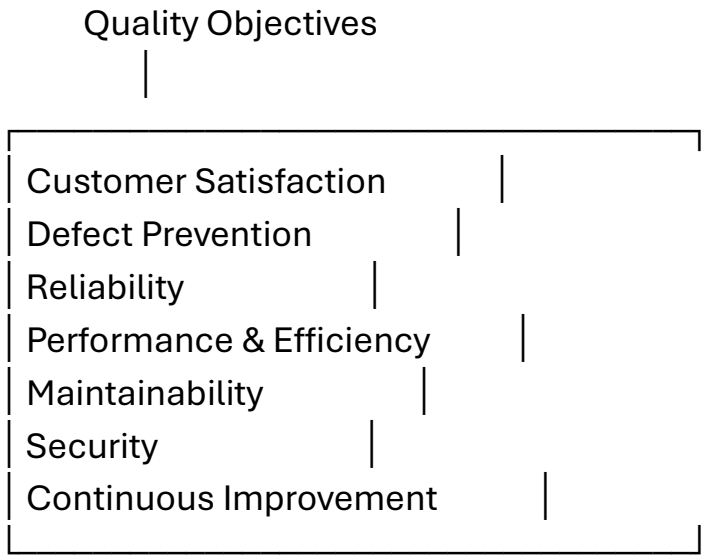
- Protect software and data from unauthorized access.
- Implement authentication, authorization, and encryption.

---

7. Continuous Improvement

- Improve software processes using quality metrics, customer feedback, and the **PDCA (Plan–Do–Check–Act)** cycle.

Diagram



Benefits

- Produces high-quality software.
- Improves customer satisfaction.
- Reduces defects and maintenance costs.
- Enhances productivity.
- Ensures continuous improvement.

Conclusion

Quality objectives help organizations deliver **reliable, secure, maintainable, and customer-oriented software**, resulting in improved software quality and business success.

Q2 (c) Plan Software Quality Control with Respect to College Attendance Software. [7 Marks]

Definition

Software Quality Control (SQC) is the process of identifying and removing defects from software through reviews, inspections, and testing to ensure that the software meets quality standards.

Software Quality Control Plan

1. Requirement Verification

- Verify all requirements such as student login, faculty login, attendance marking, report generation, and notifications.
- Ensure requirements are complete and correct.

---

2. Design Review

- Review the database schema, UI design, and software architecture.
- Ensure attendance records are stored accurately and securely.

---

3. Code Review

- Conduct peer reviews to identify coding errors.
- Follow coding standards and best practices.

---

4. Testing

Perform different testing levels:

- **Unit Testing** – Test login, attendance, and report modules.
- **Integration Testing** – Verify interaction between modules.
- **System Testing** – Test the complete attendance system.
- **Acceptance Testing** – Confirm the software meets college requirements.

---

5. Defect Management

- Record defects in a defect tracking system.
- Assign defects to developers, fix them, retest, and close them.

---

6. Performance and Security Testing

- Ensure the system supports multiple users simultaneously.
- Verify secure login and protection of attendance records.

---

7. Documentation and Maintenance

- Prepare user manuals, test reports, and maintenance documents.
- Update the software based on user feedback and changing requirements.

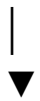
Quality Control Flow Diagram

Requirements

Design Review

Code Review

Testing
(Unit → Integration → System → Acceptance)

Defect Fixing & Retesting

Final Quality Check

▼

Software Release

Expected Quality Attributes

- Accurate attendance calculation.
- Secure user authentication.
- Fast response time.
- Reliable report generation.
- Easy maintenance.
- User-friendly interface.

Q1 (b) Differentiate Between Continuous Improvement and Continual Improvement. [7 Marks]

Definition

- **Continuous Improvement** is an **ongoing, uninterrupted** process of making improvements continuously without breaks.
- **Continual Improvement** is a process of making **incremental improvements at regular intervals**, based on evaluation and feedback.

Difference Between Continuous Improvement and Continual Improvement

Basis	Continuous Improvement	Continual Improvement
Definition	Improvement occurs continuously without interruption.	Improvement occurs repeatedly in stages or cycles.
Nature	Ongoing and uninterrupted process.	Periodic and step-by-step process.
Approach	Constant improvement every time.	Incremental improvement after evaluation.
Planning	Less emphasis on planned intervals.	Improvements are planned and reviewed regularly.
Feedback	Feedback is incorporated continuously.	Feedback is analyzed before each improvement cycle.
Example	Continuous monitoring of server performance.	Using the PDCA cycle to improve software quality after each release.
Goal	Achieve ongoing optimization.	Achieve gradual and sustainable quality improvement.

Diagram

Continuous Improvement

Improve → Improve → Improve → Improve → Improve
(Without interruption)

Continual Improvement

Plan → Do → Check → Act
  |
  ▼
Repeat Cycle
(Improvement at regular intervals)

Advantages

Continuous Improvement

- Immediate improvements.
- Faster problem detection.
- Better operational efficiency.

Continual Improvement

- Systematic and planned improvements.
- Easier performance evaluation.
- Better long-term quality management.

Q1 (c) Plan Software Quality Control with Respect to Military System. [7 Marks]

Definition

Software Quality Control (SQC) is the process of ensuring that software meets specified quality standards by detecting and correcting defects through reviews, inspections, testing, and verification.

For a **Military System**, software quality control is extremely important because the software must be **accurate, reliable, secure, and fault-tolerant**. Even a small defect can lead to mission failure or loss of life.

Software Quality Control Plan for Military System

1. Requirement Verification

- Collect and verify all military requirements.
- Ensure requirements are complete, accurate, and unambiguous.
- Define security, reliability, and performance requirements.

---

2. Design Review

- Review software architecture, database, communication modules, and security mechanisms.
- Ensure fault tolerance and backup mechanisms are included.

---

3. Code Review

- Conduct peer reviews and inspections.
- Follow secure coding standards.
- Remove coding defects before testing.

---

4. Testing

Perform different levels of testing:

- **Unit Testing** – Test each software module individually.
- **Integration Testing** – Verify communication between modules.
- **System Testing** – Test the complete military software.
- **Acceptance Testing** – Confirm that the software meets military requirements.

---

5. Security and Performance Testing

- Perform penetration testing to identify security vulnerabilities.
- Test authentication, encryption, and access control.
- Verify software performance under heavy load and critical conditions.

---

6. Defect Management

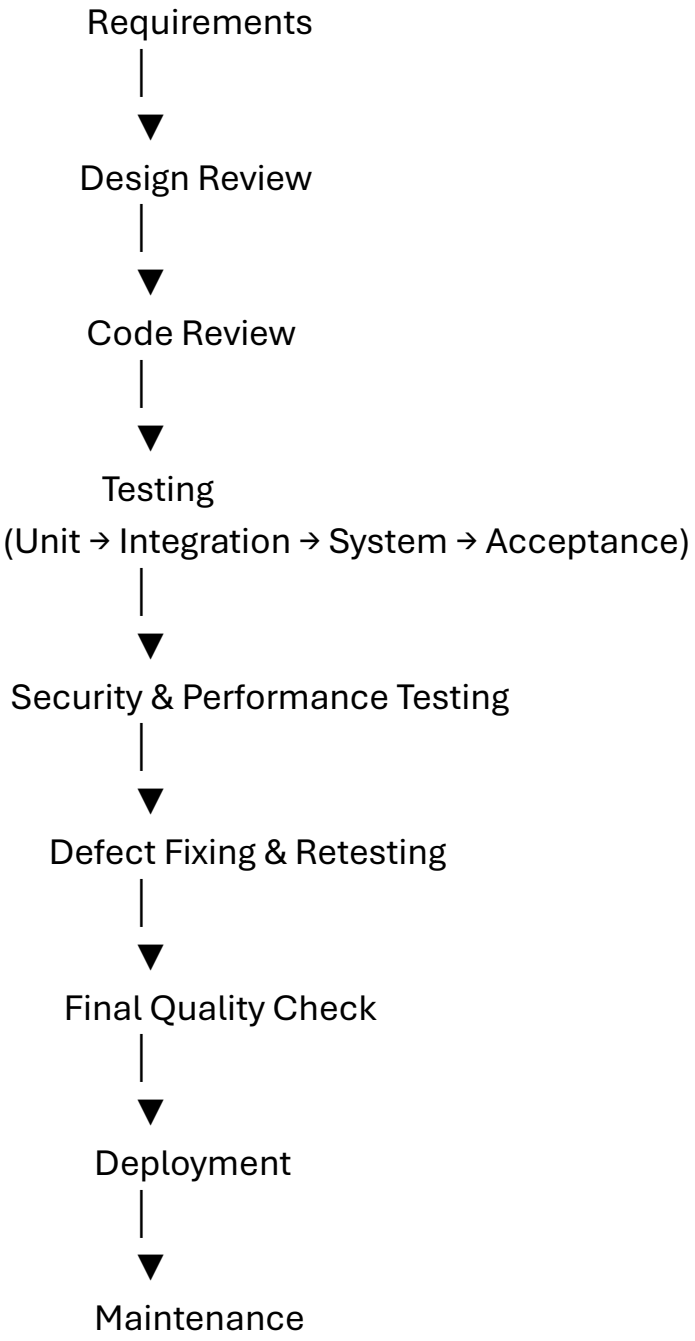
- Record all defects in a defect tracking system.
- Assign defects to developers.
- Fix, retest, and close defects before deployment.

---

7. Documentation and Maintenance

- Prepare requirement documents, design documents, test reports, and user manuals.
- Perform regular updates, bug fixes, and security patches after deployment.

Software Quality Control Flow Diagram



Quality Attributes Required in a Military System

- High Reliability
- High Security
- Accuracy
- Fault Tolerance
- High Availability
- Fast Performance
- Data Integrity
- Maintainability

Advantages

- Ensures mission success.
- Protects sensitive military information.
- Reduces software failures.
- Improves reliability and availability.
- Increases system security.
- Enhances overall software quality.

Q2 (a) List and Explain Different Characteristics of Software. [7 Marks]

Definition

Software is a collection of programs, procedures, documentation, and related data that instructs a computer to perform specific tasks. Unlike hardware, software is **intangible** and developed through engineering rather than manufacturing.

Characteristics of Software

1. Intangible

- Software cannot be seen or touched like hardware.
- It exists in the form of programs, files, and documentation.

Example: MS Word, Google Chrome.

---

2. Developed, Not Manufactured

- Software is created by writing code and designing algorithms.
- After development, it can be copied at a very low cost.

---

3. Does Not Wear Out

- Software does not physically deteriorate with use.
- It may become outdated due to changing technology or requirements, but it does not wear out like hardware.

---

4. Highly Complex

- Modern software consists of many modules and millions of lines of code.
- Higher complexity increases the chances of defects.

---

5. Easy to Modify

- Software can be updated to fix bugs, improve performance, or add new features.

- This process is known as **software maintenance**.

---

6. Mostly Custom-Built

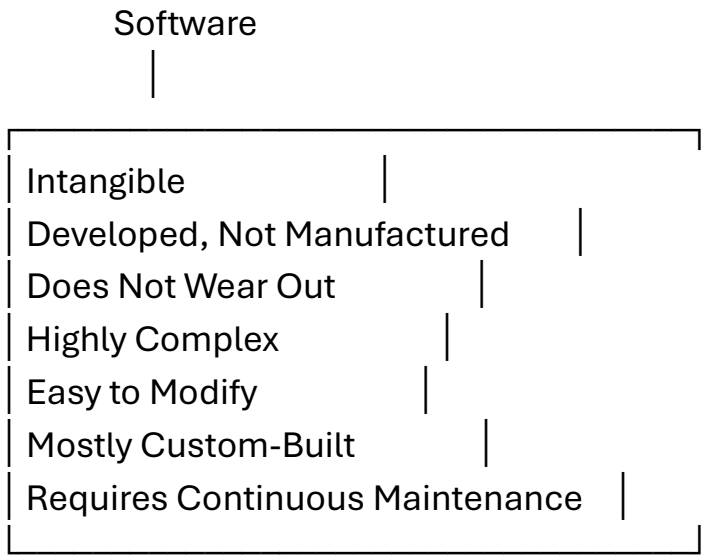
- Many software systems are developed according to the specific needs of customers or organizations.
- Some are generic products, while others are customized.

---

7. Requires Continuous Maintenance

- Software needs regular maintenance to remove defects, improve performance, and adapt to changing user requirements.

Characteristics of Software Diagram



Advantages of These Characteristics

- Easy to distribute and copy.
- Can be upgraded with new features.
- Supports automation and digital services.
- Can be customized for different users.
- Provides long-term usability through maintenance.

Q2 (b) Explain:

(i) General Requirements

(ii) Present Requirements [7 Marks]

(i) General Requirements

Definition

General Requirements are the basic requirements that every software product should satisfy to ensure it functions correctly and meets quality standards.

General Requirements

1. **Functional Requirements**
    - Specify what the software should do.
    - Example: User login, attendance marking.
2. **Performance Requirements**
    - Software should provide fast response and efficient processing.
3. **Reliability**
    - Software should perform consistently without failure.
4. **Security**
    - Protect data from unauthorized access using authentication and encryption.
5. **Usability**
    - Software should be easy to learn and use.
6. **Maintainability**
    - Software should be easy to modify, update, and debug.
7. **Portability**
    - Software should run on different hardware and operating systems.

(ii) Present Requirements

Definition

Present Requirements are the current needs and expectations of customers and organizations that software must satisfy in today's environment.

Present Requirements

1. **High Quality**
    - Software should be reliable and defect-free.
2. **Fast Delivery**
    - Software should be developed and delivered within the specified time.
3. **Cost Effectiveness**
    - Development and maintenance costs should be minimized.
4. **Security**
    - Strong protection against cyber threats and data breaches.
5. **Scalability**
    - Software should support increasing users and data.
6. **User-Friendly Interface**
    - Easy navigation and attractive design.

7. **Continuous Updates**
    - Software should support regular improvements and bug fixes.

Difference Between General and Present Requirements

General Requirements	Present Requirements
Basic quality needs	Current market and user expectations
Focus on software functions	Focus on modern technologies and user experience
Mostly technical	Technical as well as business-oriented

Conclusion

General requirements ensure the **basic quality** of software, while present requirements focus on **modern customer expectations** such as security, scalability, and fast delivery.

Q2 (c) Describe Total Quality Management (TQM) Principle of Continual Improvement. [7 Marks]

Definition of TQM

Total Quality Management (TQM) is a management approach that involves all employees in continuously improving products, services, and processes to achieve customer satisfaction.

Continual Improvement Principle

Continual Improvement means making **regular and systematic improvements** in products, services, and processes instead of making improvements only once.

Principles of Continual Improvement

1. Customer Focus

- Improve software based on customer needs and feedback.

2. Process Improvement

- Continuously improve development and testing processes.

3. Employee Participation

- Every employee contributes to quality improvement.

4. Defect Prevention

- Prevent defects instead of fixing them later.

5. Measurement and Analysis

- Use quality metrics to evaluate performance and identify improvements.

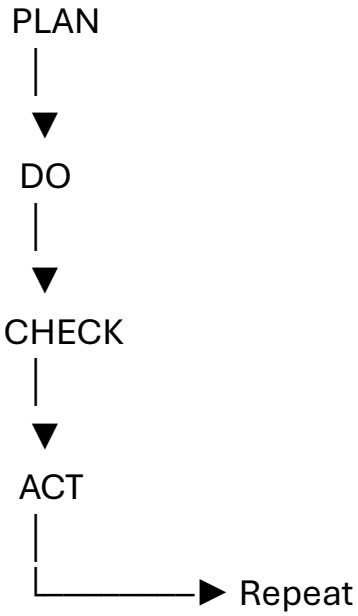
6. PDCA Cycle

- Apply the **Plan → Do → Check → Act** cycle for continuous improvement.

7. Learning and Innovation

- Learn from previous projects and adopt new technologies and best practices.

PDCA Diagram



Benefits

- Improves software quality.
- Reduces defects.
- Increases productivity.
- Lowers development cost.
- Enhances customer satisfaction.
- Supports continuous organizational growth.

Conclusion

The **Continual Improvement** principle of **TQM** encourages organizations to continuously improve their software processes using customer feedback, performance measurement, employee involvement, and the **PDCA cycle**, leading to high-quality software and greater customer satisfaction.

I analyzed the **Unit 1 syllabus** and all the **previous in-sem questions** you've shared. SPPU usually **does not repeat the exact same question** every exam. Instead, it repeats the **same concept** with different wording or a different application (e.g., college attendance → military system → space research).

Step 1: Questions Already Asked (PYQs)

Software Quality

- ✅ Define Software Quality & Core Components
- ✅ Constraints of Software Product Quality Assessment
- ❌ Characteristics of Software Quality *(not directly asked)*

Testing

- ✅ Objectives of Testing
- ✅ Testing vs Debugging
- ❌ Need of Testing *(not directly asked)*
- ❌ Quality Assurance vs Testing *(not directly asked)*

TQM

- ✅ Continual Improvement
- ✅ PDCA Cycle
- ❌ Statistical Process Control (SPC)
- ❌ Cultural Changes
- ❌ Benchmarking & Metrics
- ❌ Problem Solving Techniques
- ❌ Quality Practices of TQM

SDLC

- ✅ Draw & Explain SDLC

- ✅ Problematic Areas of SDLC
- ❌ SDLC Models
- ❌ Software Development Process

Quality Management

- ✅ Pillars of QMS
- ✅ Quality Objectives
- ✅ Quality & Productivity
- ❌ Structure of QMS
- ❌ Processes Related to Software Quality
- ❌ Software Quality Management

Product

- ✅ Types of Products
- ✅ General/Product Requirements
- ❌ Software Development Process

Application-Based Questions

- ✅ College Attendance
- ✅ Military System
- ✅ Space Research

Highly Probable Questions (★★★★★)

These are the topics from the syllabus that **have not yet appeared directly** and are therefore the strongest candidates.

1. Explain Quality Assurance (QA) and Differentiate between QA and Testing. [7]

Why?

- QA is in the syllabus.
- Testing has been asked many times.
- QA itself has never been asked.

---

2. Explain Need and Objectives of Software Testing. [7]

They asked objectives separately.

Now they may combine **Need + Objectives**.

---

3. Explain Why Software Has Errors, Defects and Failures. Discuss Causes and Effects. [7]

Very important syllabus topic.

Possible wording:
Explain why software has errors. Discuss causes and effects.

---

4. Explain Statistical Process Control (SPC). [7]
Never asked.
Very high probability.

---

5. Explain Benchmarking and Metrics. [7]
Entire topic untouched.

---

6. Explain Cultural Changes in Total Quality Management. [7]
Never appeared.

---

7. Explain Quality Practices of TQM. [7]
Very likely because only Continual Improvement has been asked.

---

8. Explain Problem Solving Techniques used in Software Quality. [7]
Completely untouched.

---

9. Explain Software Development Process. [7]
Different from SDLC.
Likely theory question.

---

10. Explain Software Quality Management (SQM). [7]
Never asked.

---

11. Explain Structure of Quality Management System (QMS). [7]
Pillars asked many times.
Structure never asked.

---

12. Explain Processes Related to Software Quality. [7]
Never appeared.

Medium Probability (★★★★☆)
Explain SDLC Models
Like

- Waterfall

- Spiral
- Prototype
- Incremental

Instead of SDLC phases.

Compare QA and QC

Very famous question.

Explain Defect, Error and Failure with Examples.

Very common university question.

Explain Customer Supplier Process Relationship.

Core Components were asked.

This is the remaining part.

Explain Historical Perspective of Software Quality.

First topic in syllabus.

Never appeared.

Application-Based Question Prediction

Every paper has one.

Already asked

- College Attendance
- Military
- Space

Possible new ones:

- Hospital Management System ⭐ ⭐ ⭐ ⭐ ⭐
- Railway Reservation System ⭐ ⭐ ⭐ ⭐
- Banking System ⭐ ⭐ ⭐ ⭐
- Airport Reservation System ⭐ ⭐ ⭐
- Online Shopping System ⭐ ⭐ ⭐
- Library Management System ⭐ ⭐ ⭐
- Smart Traffic Management ⭐ ⭐ ⭐
- E-Voting System ⭐ ⭐ ⭐ ⭐

Most Expected Long Questions

1.

Define Software Quality.

Explain Quality Management System.

2.

Explain SDLC Models.

3.

Explain QA.

Differentiate QA and QC.

4.

Explain TQM and Quality Practices.

5.

Explain SPC with Diagram.

6.

Benchmarking and Metrics.

7.

Software Errors Defects Failures.

8.

Need of Testing.

Objectives of Testing.

9.

Structure of QMS.

10.

Processes Related to Software Quality.

My Prediction (Highest Probability)

Topic	Probability
QA vs Testing	⭐ ⭐ ⭐ ⭐ ⭐

Topic	Probability
Software Errors, Defects & Failures	⭐⭐⭐⭐⭐
Statistical Process Control (SPC)	⭐⭐⭐⭐⭐
Benchmarking & Metrics	⭐⭐⭐⭐⭐
Software Quality Management	⭐⭐⭐⭐☆
Structure of QMS	⭐⭐⭐⭐☆
Processes Related to Software Quality	⭐⭐⭐⭐☆
Quality Practices of TQM	⭐⭐⭐⭐☆
Need of Testing	⭐⭐⭐⭐☆
SDLC Models	⭐⭐⭐⭐☆
Problem Solving Techniques	⭐⭐⭐☆☆
Historical Perspective	⭐⭐☆☆☆

What to study if you have limited time

If you can prepare only **10 topics**, I recommend:

1. Software Quality
2. SDLC & SDLC Models
3. QA vs Testing
4. Testing vs Debugging
5. Software Errors, Defects & Failures
6. TQM + PDCA + Quality Practices
7. Statistical Process Control (SPC)
8. Benchmarking & Metrics
9. Quality Management System (Structure + Pillars)
10. Application-based Software Quality Control (Hospital/Banking/Library)

These topics cover almost all of Unit 1 and align closely with SPPU's pattern of rotating concepts rather than repeating identical questions. Based on the PYQs and the untouched syllabus areas, preparing these gives you the strongest chance of covering what is likely to appear next.